

# Como colocar um sistema no banco de dados

*Passo a passo simples, sem jargão, pra quem nunca fez.  
Serve pra qualquer sistema Python que hoje guarda dados em  
arquivos locais (JSON, CSV, SQLite) e precisa passar pra um  
banco compartilhado entre várias máquinas.*

Estimativa: 2 a 4 horas na primeira vez.

O que você vai fazer:

- Escolher o servidor
- Instalar o banco (MariaDB)
- Criar o usuário e o banco vazio
- Liberar acesso pela rede
- Criar as tabelas do sistema
- Migrar os dados que já existiam nos arquivos locais
- Testar de outra máquina
- Configurar cada máquina cliente
- Fazer backup automático

## Passo 1 — Escolher o servidor

O que é: uma máquina que fica sempre ligada, na rede do escritório, e vai guardar os dados que todo mundo compartilha.

Pode ser:

- Um computador antigo que fica só pra isso (mais barato).
- Um NAS (Synology, QNAP) — quase todos têm MariaDB pronto.
- Uma máquina virtual num servidor Windows Server.
- Um serviço na nuvem (AWS RDS, DigitalOcean, etc) — mais caro, mas não precisa cuidar de hardware.

Anotar antes de começar:

- IP interno do servidor (ex: 10.0.1.47). Descubra com ipconfig no cmd da máquina servidor.
- Sistema operacional (Windows, Linux).

## Passo 2 — Instalar o banco (MariaDB)

Por que MariaDB: é grátis e sem limite, todos os operadores conseguem acessar ao mesmo tempo, é rápido pra dados de escritório (até milhões de linhas sem suar).

### 2.1 Baixar

Site oficial: [mariadb.org/download](https://mariadb.org/download). Windows: escolha o .msi de 64 bits. Linux: geralmente já vem no gerenciador de pacotes (apt install mariadb-server no Ubuntu).

### 2.2 Instalar (Windows)

- Duplo-clique no instalador. Aceite os termos → Next.
- Deixe o caminho padrão.

- Na tela Default instance properties: crie uma senha forte pro root e anote em lugar seguro.
- NÃO marque 'Enable access from remote machines' ainda — vamos configurar isso do jeito seguro no Passo 4.
- Porta: pode deixar 3306, ou trocar pra 3307 se já tiver outro banco na 3306.
- Próxima → Install → Finish.

## 2.3 Testar

Abra o cmd na máquina servidor:

```
mysql -u root -p
```

Digite a senha do root. Se aparecer MariaDB [(none)]>, deu certo. Digite exit pra sair.

## Passo 3 — Criar o usuário e o banco vazio

Por que: o usuário root é só pra administrar. O sistema vai usar um usuário próprio, com acesso só ao banco dele — mais seguro.

### 3.1 Entrar no MariaDB

```
mysql -u root -p
```

### 3.2 Rodar os comandos (uma linha por vez)

Copia e cola. Troca NOMEDOSISTEMA pelo nome curto do seu sistema (ex: conciliador, financeiro, estoque). Escolha uma senha forte pra SENHAFORTE — anote.

```
CREATE DATABASE NOMEDOSISTEMA CHARACTER SET utf8mb4
COLLATE utf8mb4_unicode_ci;

CREATE USER 'NOMEDOSISTEMA_master'@'%' IDENTIFIED BY
'SENHAFORTE';

GRANT ALL PRIVILEGES ON NOMEDOSISTEMA.* TO
'NOMEDOSISTEMA_master'@'%';

FLUSH PRIVILEGES;
EXIT;
```

O que cada linha faz:

- Linha 1: cria o banco vazio.
- Linha 2: cria o usuário. O '%' significa 'pode conectar de qualquer máquina da rede'.
- Linha 3: dá permissão total pra esse usuário só no banco dele.
- Linha 4: aplica as mudanças imediatamente.
- Linha 5: sai.

## Passo 4 — Liberar acesso pela rede

Sem isso, só a máquina servidor consegue conectar. Vamos abrir pra rede local.

### 4.1 Editar a configuração do MariaDB

Windows: abre com o Bloco de Notas o arquivo:

```
C:\Program Files\MariaDB 11.x\data\my.ini
```

Linux:

```
sudo nano /etc/mysql/mariadb.conf.d/50-server.cnf
```

Procure a linha que começa com bind-address. Deve estar tipo:

```
bind-address = 127.0.0.1
```

Troque pra:

```
bind-address = 0.0.0.0
```

Isso diz 'aceita conexão de qualquer IP', não só da própria máquina. Salva e fecha.

## 4.2 Reiniciar o MariaDB

Windows: abre 'Serviços' (Win+R → services.msc), procura 'MariaDB', botão direito → Reiniciar.

Linux:

```
sudo systemctl restart mariadb
```

## 4.3 Liberar a porta no Firewall

Windows:

- Abre 'Windows Defender Firewall com Segurança Avançada'.
- Regras de Entrada → Nova Regra.
- Tipo: Porta → TCP → Porta específica: 3306 (ou 3307 se trocou).
- Ação: Permitir a conexão.
- Perfil: marque Domínio + Privada (rede local). Desmarque Pública (segurança).
- Nome: MariaDB. Concluir.

Linux (com ufw):

```
sudo ufw allow 3306/tcp
```

# Passo 5 — Criar as tabelas do sistema

Cada sistema tem uma estrutura de tabelas diferente. Você precisa escrever um script Python que cria essas tabelas.

## 5.1 Modelo pronto (adaptar)

No Conciliador, esse script é o setup\_db.py. Use-o como referência — só troque o SQL das tabelas pelas suas. Estrutura básica do script:

```
import pymysql

CONFIG = {
    'host': '10.0.1.47',
    'port': 3306,
    'user': 'NOMEDOSISTEMA_master',
    'password': 'SENHAFORTE',
    'database': 'NOMEDOSISTEMA',
    'charset': 'utf8mb4',
}

TABELAS = [
    """
    CREATE TABLE IF NOT EXISTS usuarios (
        id INT AUTO_INCREMENT PRIMARY KEY,
        username VARCHAR(50) UNIQUE NOT NULL,
        senha_hash VARCHAR(200) NOT NULL
    ) ENGINE=InnoDB
    """,
]
```

```
conn = pymysql.connect(**CONFIG)
with conn.cursor() as cur:
    for sql in TABELAS:
        cur.execute(sql)
conn.commit()
conn.close()
```

## 5.2 Rodar o script

Numa máquina que já tem Python + acesso ao servidor:

```
pip install pymysql
python setup_db.py
```

*Dica: o CREATE TABLE IF NOT EXISTS só cria se não existir. Pode rodar o script quantas vezes quiser sem quebrar dados existentes.*

## Passo 6 — Migrar os dados que já existiam

Se o sistema já rodava com arquivos locais (JSON, CSV), a gente precisa mover esses dados pro banco.

Estratégia:

- Abrir o arquivo local (JSON, por exemplo).
- Pra cada registro, executar um INSERT no banco.
- Deixar o arquivo antigo como backup (não apagar).

Exemplo de script:

```
import json
import pymysql

with open('config.json', 'r', encoding='utf-8') as f:
    dados_antigos = json.load(f)

conn = pymysql.connect(**CONFIG)
with conn.cursor() as cur:
    for chave, valor in dados_antigos.items():
        cur.execute(
            'INSERT INTO app_config (chave, valor) VALUES (%s, %s)',
            (chave, json.dumps(valor)),
        )
conn.commit()
conn.close()
```

### Backup antes:

Antes de rodar, faz uma cópia dos dados antigos:

```
copy C:\SistemaAntigo\data\config.json
    C:\SistemaAntigo\data\config.json.bak
```

## Passo 7 — Testar de outra máquina

Vá pra uma máquina que não é o servidor e tem Python instalado.

### 7.1 Instalar o driver

```
pip install pymysql
```

### 7.2 Rodar um teste

Criar um arquivo teste.py:

```
import pymysql

conn = pymysql.connect(
    host='10.0.1.47',
    port=3306,
    user='NOMEDOSISTEMA_master',
    password='SENHAFORTE',
    database='NOMEDOSISTEMA',
)
with conn.cursor() as cur:
    cur.execute('SELECT VERSION()')
    print('Conectado:', cur.fetchone()[0])
conn.close()
```

Se aparecer Conectado: 10.x.x-MariaDB, funcionou.

### 7.3 Se der erro

- 'Can't connect to server': firewall bloqueando. Volta no passo 4.3.
- 'Access denied for user': senha errada ou usuário criado com 'localhost' em vez de '%'. Volta no passo 3.2 e recria com '%'.  
• 'Unknown database': o nome do banco no CONFIG está diferente do que criou. Confere.

## Passo 8 — Configurar cada máquina cliente

Em cada computador que vai usar o sistema:

### 8.1 Instalar Python e o sistema

Do jeito normal do sistema (git clone, pip install, etc). Ver o INSTALACAO\_CLIENTE.md como exemplo.

### 8.2 Colocar as credenciais

Cria o arquivo de configuração (no Conciliador é data/db\_config.json):

```
{
  "host": "10.0.1.47",
  "port": 3306,
  "user": "NOMEDOSISTEMA_master",
  "password": "SENHAFORTE",
  "database": "NOMEDOSISTEMA",
  "charset": "utf8mb4"
}
```

*Atalho prático: copia o arquivo de uma máquina já configurada via rede — em vez de digitar de novo em cada uma.*

## Passo 9 — Fazer backup automático

Sem backup você corre risco de perder tudo se o disco do servidor pifar.

### 9.1 Script simples de backup (Windows)

Criar backup.bat na máquina servidor:

```
@echo off
set DATA=%date:~6,4%-%date:~3,2%-%date:~0,2%
"C:\Program Files\MariaDB 11.x\bin\mysqldump" ^
-u root -pSENHAROOT ^
NOMEDOSISTEMA > "C:\Backups\NOMEDOSISTEMA_%DATA%.sql"
```

## 9.2 Agendar

- Abrir Agendador de Tarefas (Windows) → Criar Tarefa Básica.
- Nome: 'Backup diário do sistema'.
- Disparador: Diariamente → 23:00 (fora do expediente).
- Ação: Iniciar um programa → escolha o backup.bat.
- Concluir.

## 9.3 Manter só os últimos 30 dias

Adiciona no fim do backup.bat:

```
forfiles /P "C:\Backups" /M *.sql /D -30 /C "cmd /c del @path"
```

## 9.4 IMPORTANTE — copiar backup pra outro lugar

O backup na mesma máquina que tem o banco não vale nada se a máquina inteira quebrar. Configure cópia pra:

- Outra máquina da rede (via robocopy num .bat).
- Um NAS.
- Nuvem (OneDrive, Dropbox, Google Drive).

## Checklist final

Antes de considerar o serviço no ar:

- MariaDB instalado no servidor
- Usuário do sistema criado, com senha forte anotada
- Banco vazio criado
- Acesso pela rede liberado (bind-address + firewall)
- Tabelas criadas rodando o setup\_db.py
- Dados antigos migrados (se havia)
- Teste de outra máquina funcionou
- Pelo menos 1 máquina cliente configurada e testada
- Backup automático rodando
- Backup sendo copiado pra outro lugar
- Senha do root e do usuário anotadas em lugar seguro
- Documento de como fazer isso guardado (esse arquivo!)

## Se precisar de ajuda

- Documentação MariaDB: [mariadb.com/kb/](https://mariadb.com/kb/)
- PyMySQL: [pymysql.readthedocs.io](https://pymysql.readthedocs.io)
- Referência real desse fluxo: pasta do Conciliador (setup\_db.py, db.py, INSTALACAO\_CLIENTE.md).